STM32基础 - AD转换与DMA

课程目标

1. 熟悉ADC模数转换器工作过程，掌握非扫描模式和扫描模式的工作区别
2. 熟悉DMA原理，掌握ADC和DMA共同协作完成AD转换的工作过程

课程试验

1. 利用STM32完成AD单通道试验
2. 利用STM32完成AD多通道试验
3. 利用STM32完成DMA+AD多通道试验

课堂引入

现在随着国家工业4.0的推进，我们看到工厂中出现了越来越多智能机器人替代工人工作，比如汽车生产线上我们可以看到一个个巨型的机械臂灵巧而又高效的完成一个个工件的自动加工任务。那么这些机械臂如何知道自己当前的状态，又如何确保自己每一步的动作都刚好到位呢?

这就需要硬件电路不断的对机械臂的位置，角度等基本参数做检测，然后在不断地去修正其运行的误差，才能实现最后的分毫不差的结果。硬件电路如何对其关注的必要IO端口进行自动化测量，然后又如何把这些测量到的模拟数据变换为CPU可以识别处理的数字量呢？这就是我们这次课要解决的问题。

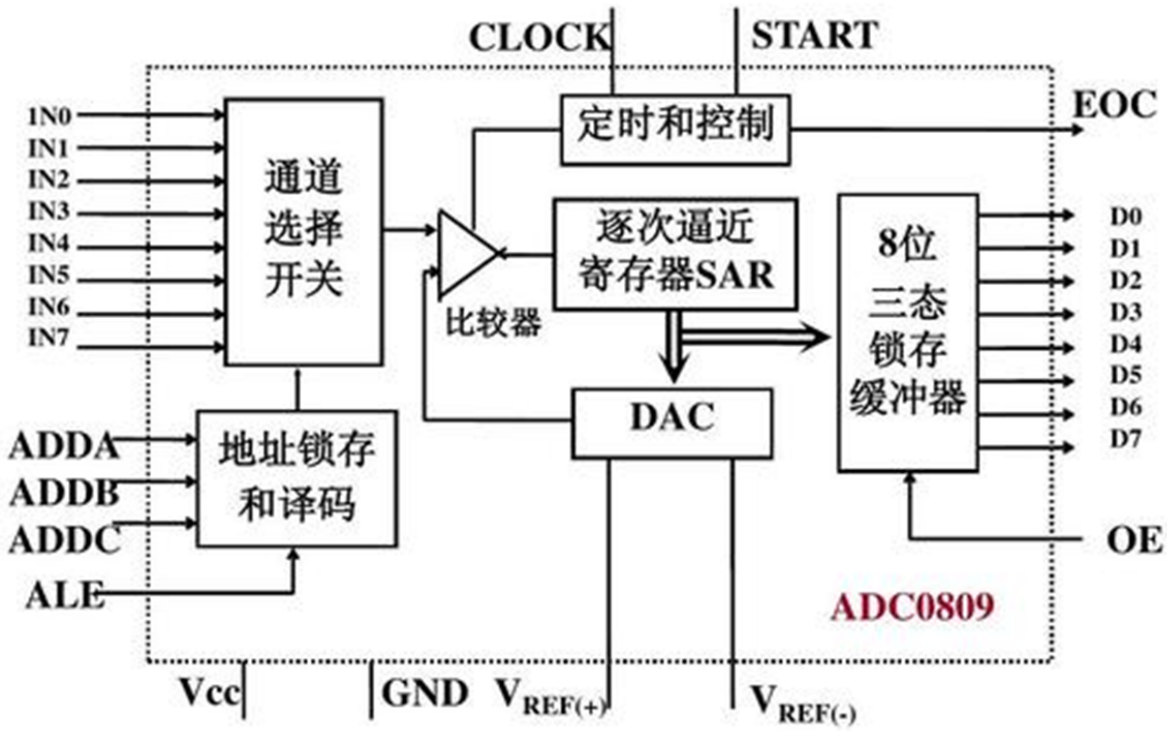
授课进程

一、AD模数转换

1、基本介绍

ADC（Analog-Digital Converter）模拟-数字转换器，简称模数转换器，有时也叫AD转换器。ADC可以将引脚上连续变化的模拟电压转换为内存中存储的数字变量，建立模拟电路到数字电路的桥梁。STM32的ADC是12位逐次逼近型ADC，1us转换时间。输入电压范围是0~3.3V，转换结果范围是0~4095。有18个输入通道，可测量16个外部和2个内部信号源。外部信号源就是16个GPIO口，在引脚上直接接模拟信号就行，不需要任何额外的电路，引脚就直接能测电压。两个内部信号源是内部温度传感器和内部参考电压，温度传感器可以测量CPU的温度。内部参考电压是一个1.2伏左右的基准电压，它是不随外部供电电压变化而变化的。如果芯片的供电不是标准的3.3伏，测量外部引脚的电压可能就不对，这时就可以读取这个基准电压进行校准，就能得到正确的电压值了。STM32的ADC具有用于常规使用的规则组和用于突发事件的注入组两个转换单元。具有模拟看门狗可以自动监测输入电压范围。STM32F103C8T6芯片的ADC资源有ADC1、ADC2两个外设，10个外部输入通道也就是它最多只能测量10个外部引脚的模拟信号，前面提到的16个外部信号源是STM32最多16个，但是这个芯片型号只有10个。

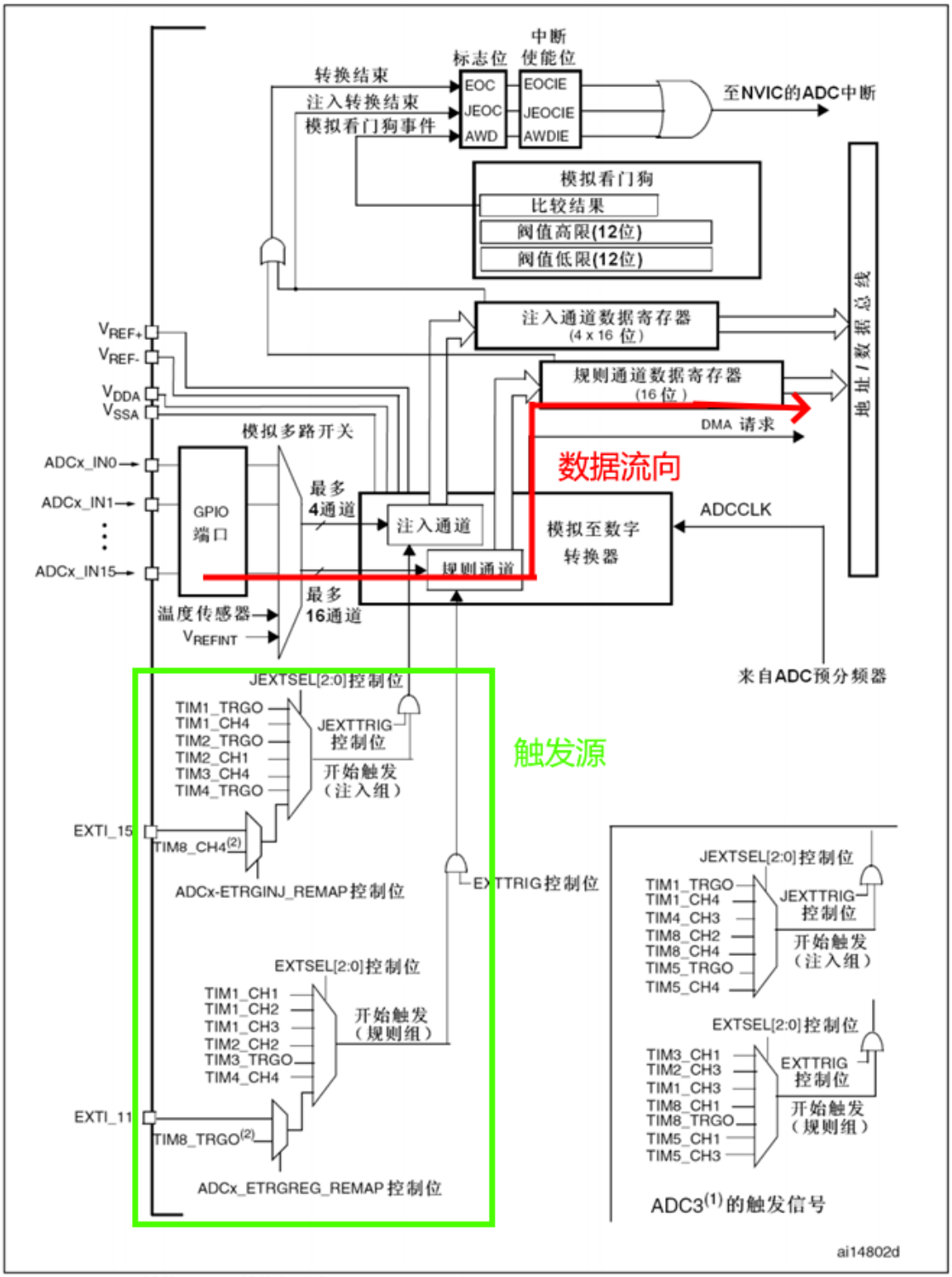
2、逐次逼近型ADC例子（ADC0809芯片）



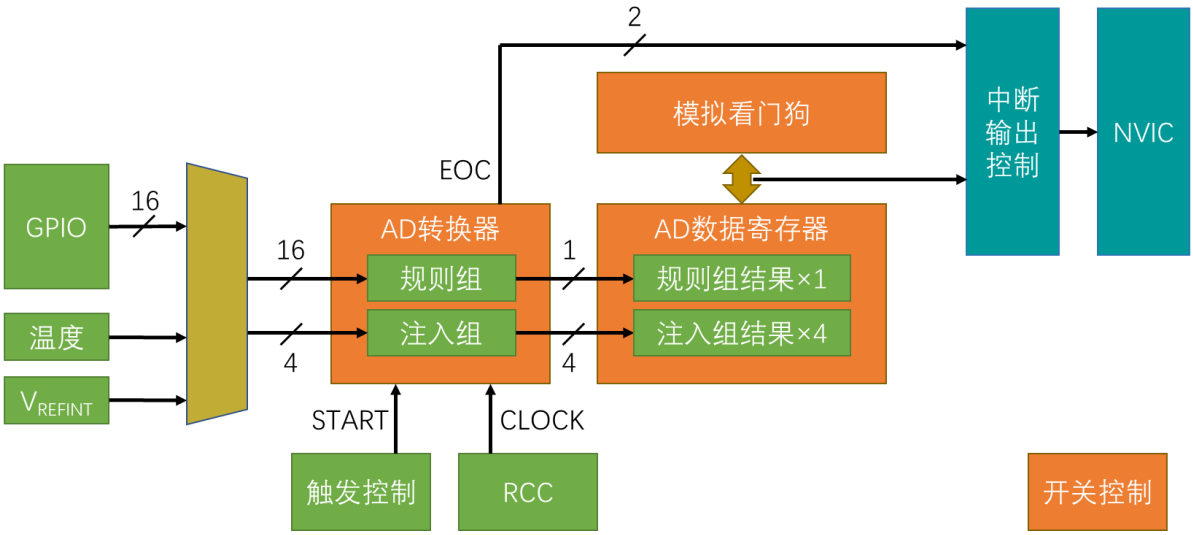
- SMT32最多情况
  - 有18个输入通道
    - 16个外部（GPIO口）
    - 2个内部信号源
      - 内部温度传感器
      - 内部参考电压（1.2伏左右的基准电压，不随外部供电电压变化而变化的）
- STM32F103C8T6芯片的ADC
  - 资源有ADC1、ADC2两个外设
  - 10个外部输入通道
- STM32 有AD，没有DA（用的是PWM）
  - ADC12共用一个GPIO口。

3、ADC框图

图24　　单个ADC框图



4、ADC基本结构



ADC基本结构图。左边是输入通道，16个GPIO口，外加两个内部通道。然后进入AD转换器，AD转换器里有两个组，一个是规则组，一个是注入组。规则组最多可以选中16个通道，注入组最多可以选择4个通道。转换的结果可以存放在AD数据寄存器里，其中规则组只有一个数据寄存器，而注入组有4个。在AD转换器下方里有触发控制，提供了开始转换这个START信号。触发控制可以选择软件触发和硬件触发。硬件触发主要是来自于定时器，当然也可以选择外部中断的引脚。右边是来自于RCC的ADC时钟CLOCK，ADC逐次比较的过程就是由这个时钟推动的。在ADC数据寄存器上面可以布置一个模拟看门狗，用于监测转换结果的范围。如果超出设定的阈值，就通过中断输出控制向NVIC申请中断。另外规则组和注入组转换完成后，会有一个EOC信号，它会置一个标志位通向NVIC。在右下角有一个开关控制，这个开关控制在库函数中就是ADC_CMD函数，用于给ADC上电的。

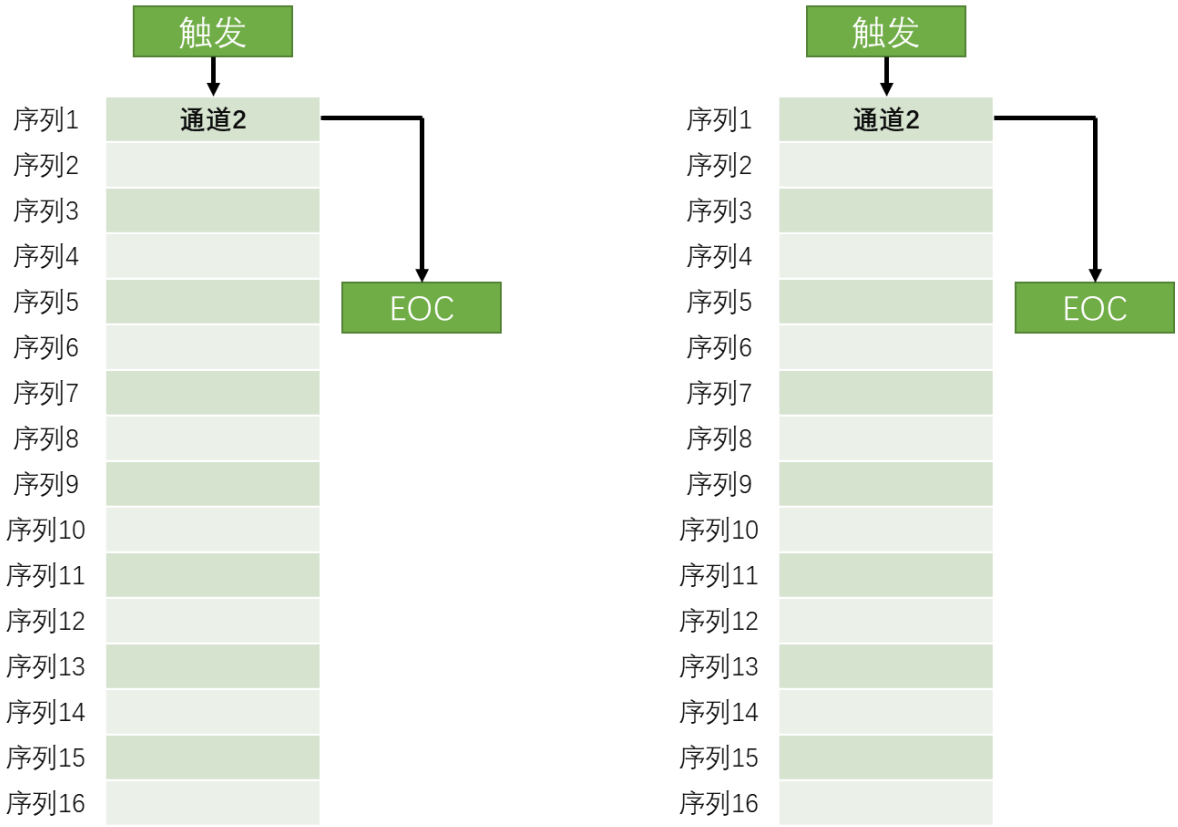
5、输入通道

这是ADC通道和STM32引脚复用的关系对应表。

通道	ADC1	ADC2	ADC3
通道0	PA0	PA0	PA0
通道1	PA1	PA1	PA1
通道2	PA2	PA2	PA2
通道3	PA3	PA3	PA3
通道4	PA4	PA4	PF6
通道5	PA5	PA5	PF7
通道6	PA6	PA6	PF8
通道7	PA7	PA7	PF9
通道8	PB0	PB0	PF10
通道9	PB1	PB1	
通道10	PC0	PC0	PC0
通道11	PC1	PC1	PC1
通道12	PC2	PC2	PC2
通道13	PC3	PC3	PC3
通道14	PC4	PC4	
通道15	PC5	PC5	
通道16	温度传感器		
通道17	内部参考电压		

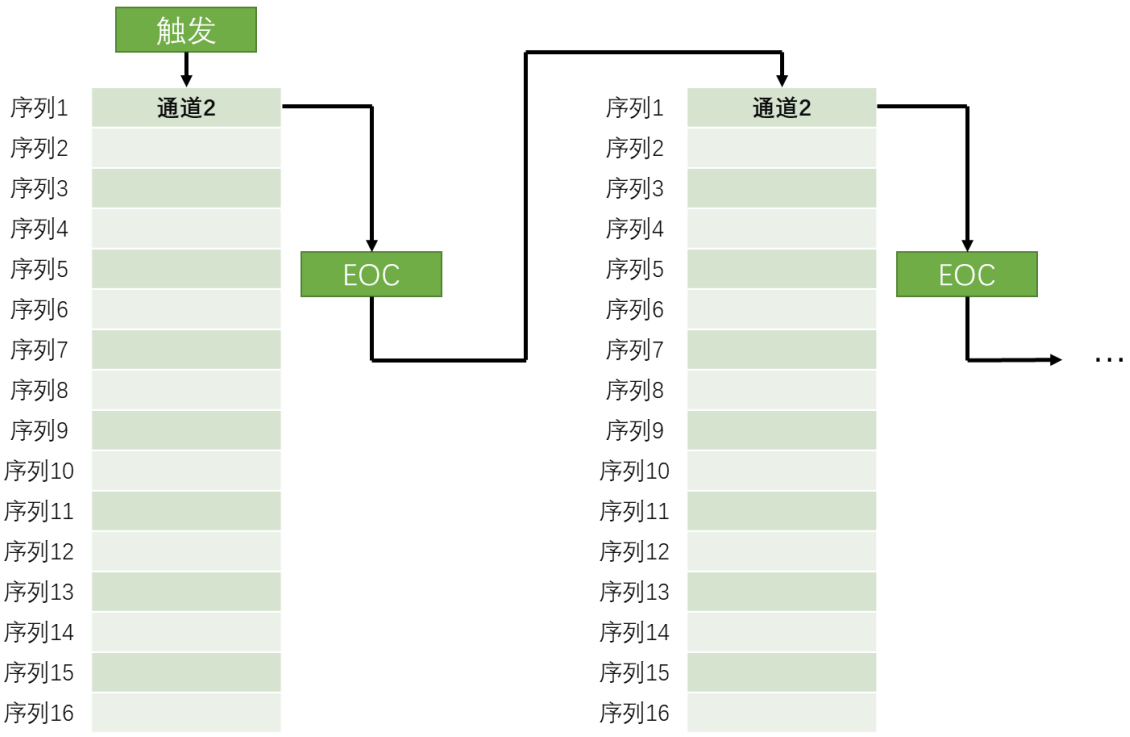
6、转换模式

（1）单次转换，非扫描模式



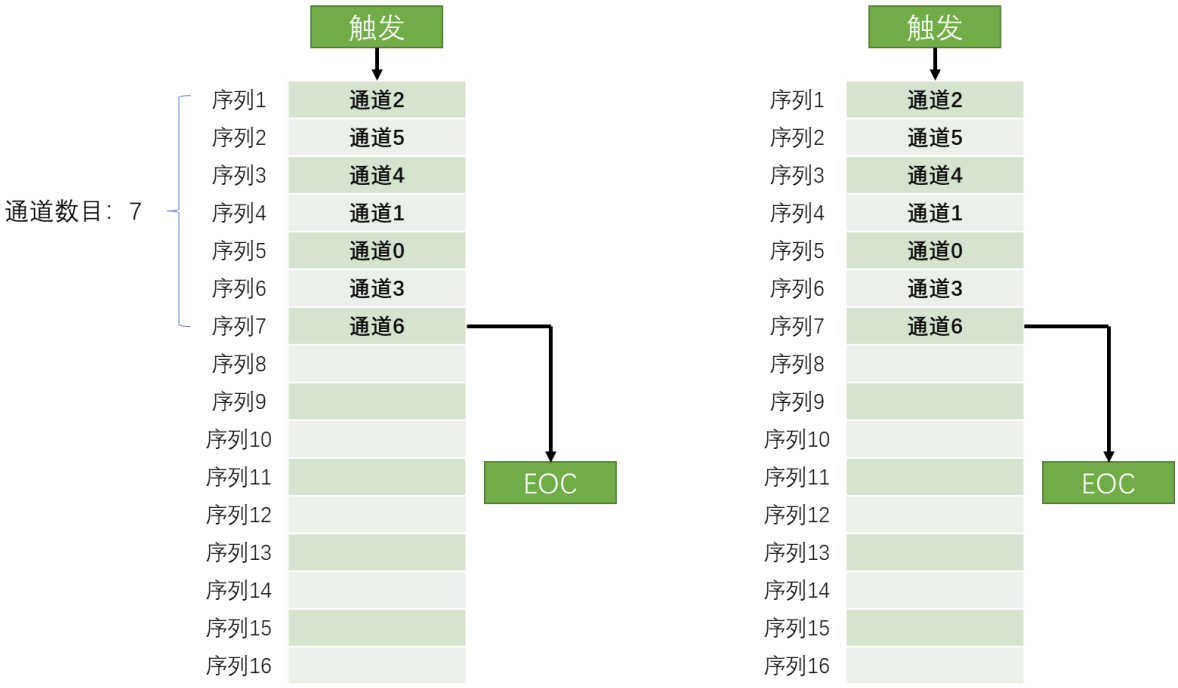
这个表就是规则组里的菜单，有16个空位，分别是序列1到序列16。可以在这里写入要转换的通道。在非扫描的模式下，只有第一个序列1的位置有效，这时同时选中一组的方式就简化为选中一个的方式了。我们可以在序列1的位置指定想转换的通道，比如在这个位置写入通道2，就可以触发转换，ADC就会对这个通道2进行模数转换，过一小段时间后转换完成，转换结果放在数据寄存器里，同时给EOC标志位置1，整个转换过程就结束了。判断这个EOC标志位，如果转换完了，就可以在数据寄存器里读取结果了。如果想再启动一次转换，那就需要再触发一次转换，结束置EOC标志位读结果。如果想换一个通道转换，那在转换之前，把第一个位置的通道2改成其他通道，然后再启动转换就行了。

（2）连续转换，非扫描模式



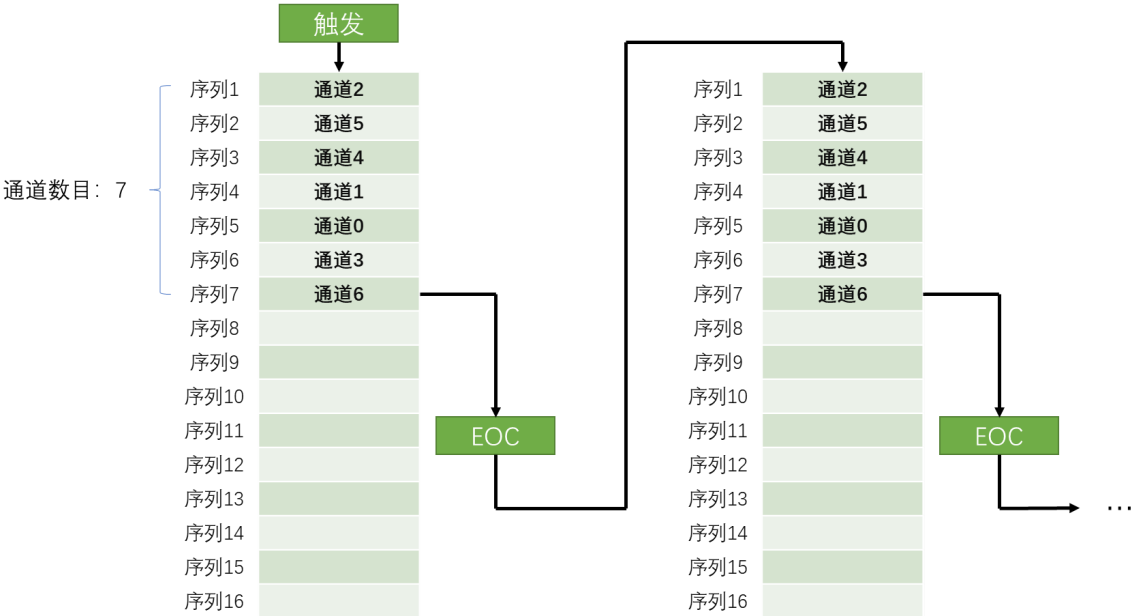
首先它还是非扫描模式，所以菜单列表就只用第一个。然后它与上一种单次转换不同的是，它在一次转换结束后不会停止，而是立刻开始下一轮的转换，然后一直持续下去。这样就只需要最开始触发一次，之后就可以一直转换了。这个模式的好处就是开始转换之后，不需要等待一段时间的，因为它一直都在转换，所以就不需要手动开始转换了，也不用判断是否结束。想要读AD值的时候，直接从数据寄存器取就是了。

（3）单次转换，扫描模式



这个模式也是单次转换，所以每触发一次转换结束后，就会停下来，下次转换就得再触发才能开始。由于它是扫描模式，这就会用到这个列表了，可以在这个列表中添加通道。比如第一个是通道2，第二个是通道5，第三个是通道4……等等。这里每个位置是通道几，可以任意指定，并且也是可以重复的。可以指定通道使用个数，比如这里指定通道数目为7，那就只会看前7个位置，在每次触发之后，ADC就依次对这前7个位置进行AD转换，转换结果都放在数据寄存器里，这里为了防止数据被覆盖，就需要用DMA及时将数据挪走，当7个通道
转换完成之后，产生EOC信号转换结束，然后再触发，下一次就又开始新一轮的转换。

（4）连续转换，扫描模式



这个模式就是在前一个模式基础上变了一点，就是一次转换完成后，立刻开始下一次的转换，其他都和上一个模式是一样的。

7、触发控制

表64　ADC1和ADC2用于规则通道的外部触发

触发源	类型	EXTSEL[2:0]
TIM1_CC1事件	来自片上定时器的内部信号	000
TIM1_CC2事件		001
TIM1_CC3事件		010
TIM2_CC2事件		011
TIM3_TRGO事件		100
TIM4_CC4事件		101
EXTI线11/TIM8_TRGO事件(1)(2)	外部引脚/来自片上定时器的内部信号	110
SWSTART	软件控制位	111

8、数据对齐

(转换结果是12位，结果寄存器是16位的)

（1）数据右对齐

规则组

0	0	0	0	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0

（2）数据左对齐

规则组

D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0	0	0	0	0

9、转换时间

AD转换的步骤：采样，保持，量化，编码

STM32 ADC的总转换时间为：

$$T_{CONV} = \text{采样时间} + 12.5\text{个}ADC\text{周期}$$

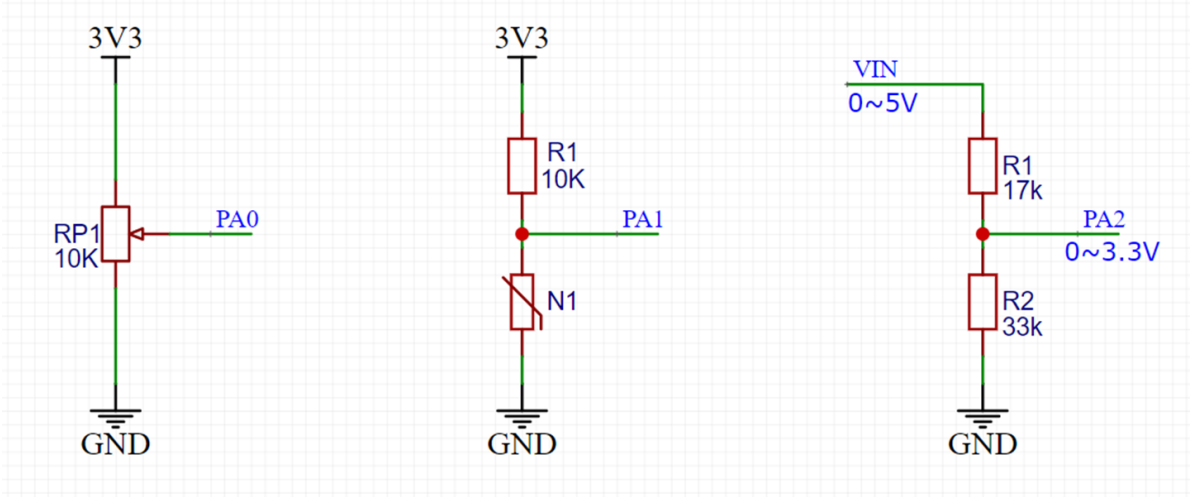
例如：当ADCCLK=14MHz，采样时间为1.5个ADC周期

$$T_{CONV} = 1.5 + 12.5 = 14\text{个}ADC\text{周期} = 1\mu s$$

10、校准

ADC有一个内置自校准模式。校准可大幅减小因内部电容器组的变化而造成的准精度误差。校准期间，在每个电容器上都会计算出一个误差修正码(数字值)，这个码用于消除在随后的转换中每个电容器上产生的误差。建议在每次上电后执行一次校准。注意启动校准前， ADC必须处于关电状态超过至少两个ADC时钟周期。

11、硬件电路



第一个电路是电位器产生一个可调的电压，这里电位器的两个固定端，一端接3.3伏，另一端接地。这样中间的滑动端就可以输出一个0-3.3伏可调的电压。这里可以接ADC的输入通道，比如PA0口。当滑动端往上滑时电压增大，往下滑时电压减小。另外注意这个电阻的阻值不要给太小，因为这个电阻两端是直接跨接在电源正负极的，如果阻值太小，那这个电阻就会比较费电，如果再小就有可能发热冒烟了。因此一般至少要接千欧级的电阻，比如这里接的是10K的电阻，这是电位器产生可调电压的电阻。

中间的电路是传感器输出电压的电路，一般来说像光敏电阻、热敏电阻、红外接收管、麦克风等等，都可以等效为一个可变电阻。其电阻阻值没法直接测量，所以就可以通过和一个固定电阻串联分压，来得到一个反映电阻值电压的电路。当传感器阻值变小时，下拉作用变强，输出端电压就下降；传感器阻值变大时，下拉作用变弱，输出端受上拉电阻的作用，电压就会升高。这个固定电阻一般可以选择和传感器阻值相近的电阻，这样可以得到一个位于中间电压区域，比较好的输出。这里传感器和固定电阻的位置，也可以换过来，这样的话输出电压的极性就反过来了。这就是分压方法来输出传感器阻值的电路。

第三个电路是一个简单的电压转换电路，比如想测一个0-5伏的VIN电压，但是ADC只能接收0-3.3伏的电压。就可以搭建一个这样的简易转换电路。这里还是使用电阻进行分压，上面阻值17K，下面阻值33K，加一起是50K。所以根据分压公式计算中间的电压就是VIN除以50K乘以33K，最后得到的电压范围就是0-3.3伏，就可以进入ADC转换了。如果你想采集5伏、10伏这些电压的话，可以使用这个电路。但是如果电压再高一些，就不建议使用这个电路了，那样可能会比较危险。高电压采集最好使用一些专用的采集芯片，比如隔离放大器等等，做好高低电压的隔离，保证电路的安全。

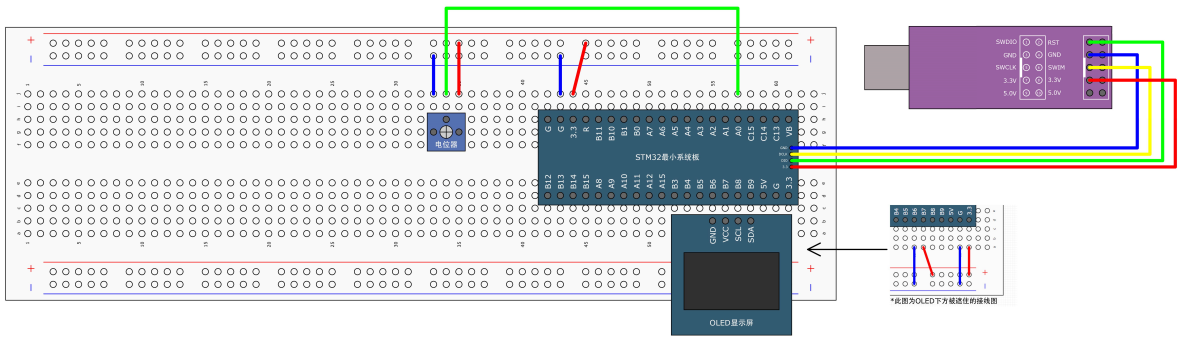
二、AD转换实验

1、AD单通道单次转换

（1）实验目的

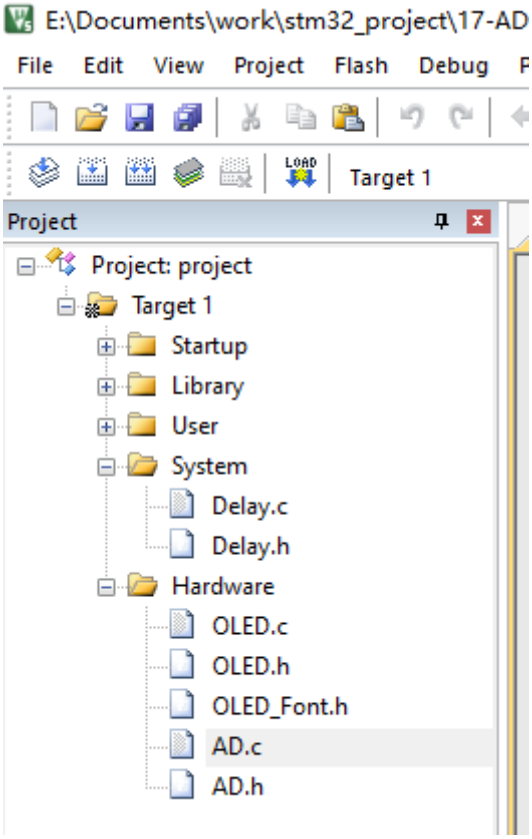
通过控制STM32读取因电位器调整引起输入电压变化模拟量，在OLED上显示AD转换后的数字值。

（2）硬件电路接线图



（3）程序设计

工程结构如下图：



其中Hardware组与之前工程不一样，这个组主要增加了AD相关驱动代码。具体代码展示如下：

AD.c

```
#include "stm32f10x.h"                  // Device header

void AD_Init(void) {
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_ADC1, ENABLE);    // 打开ADC1的时钟
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);   // 打开GPIOA的时钟

    RCC_ADCCLKConfig(RCC_PCLK2_Div6);   // 配置ADC时钟，选择时钟6分频，ADCCLK = 72MHz / 6 =
12MHz

    GPIO_InitTypeDef GPIO_InitStructure;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AIN;   // 配置模拟输入模式
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_0;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    // 配置规则组通道，规则组序列1的位置，配置为通道0
    ADC_RegularChannelConfig(ADC1, ADC_Channel_0, 1, ADC_SampleTime_55Cycles5);

    ADC_InitTypeDef ADC_InitStructure;
    ADC_InitStructure.ADC_ContinuousConvMode = DISABLE;     // 配置非扫描模式
    ADC_InitStructure.ADC_DataAlign = ADC_DataAlign_Right;  // 配置数据右对齐
    ADC_InitStructure.ADC_ExternalTrigConv = ADC_ExternalTrigConv_None;     // 配置使用软
件触发
    ADC_InitStructure.ADC_Mode = ADC_Mode_Independent;      // 配置使用独立模式，单独使用
ADC1
    ADC_InitStructure.ADC_NbrOfChannel = 1;     // 配置通道数为1
    // 单扫描模式
```

```
    ADC_InitStructure.ADC_ScanConvMode = DISABLE;
    // 连续扫描模式
    // ADC_InitStructure.ADC_ScanConvMode = ENABLE;
    ADC_Init(ADC1, &ADC_InitStructure);     // 配置ADC初始化

    ADC_Cmd(ADC1, ENABLE);      // 使能ADC

    // ADC校准
    ADC_ResetCalibration(ADC1);
    while (ADC_GetResetCalibrationStatus(ADC1) == SET);
    ADC_StartCalibration(ADC1);
    while (ADC_GetCalibrationStatus(ADC1) == SET);

    // 连续扫描需要配置下面这句触发语句
    // ADC_SoftwareStartConvCmd(ADC1, ENABLE);
}

uint16_t AD_GetValue(void) {
    // 单扫描模式
    ADC_SoftwareStartConvCmd(ADC1, ENABLE);
    while (ADC_GetFlagStatus(ADC1, ADC_FLAG_EOC) == RESET);
    // 连续扫描不需要上面两行代码
    return ADC_GetConversionValue(ADC1);
}
```

AD.h

```
#ifndef __AD_H
#define __AD_H

void AD_Init(void);
uint16_t AD_GetValue(void);

#endif
```

main.c

```
#include "stm32f10x.h"                  // Device header
#include "Delay.h"
#include "OLED.h"
#include "AD.h"

uint16_t ADValue;
float Voltage;

int main(void){

    OLED_Init();
    AD_Init();
```

```
    OLED_ShowString(1, 1, "ADValue:");
    OLED_ShowString(2, 1, "Voltage:0.00V");

    while(1){
        ADValue = AD_GetValue();        // 获取AD转换的值
        Voltage = (float)ADValue / 4095 * 3.3;      // 将AD值线性变换到0~3.3的范围，表示电压

        OLED_ShowNum(1, 9, ADValue, 4);     // 显示AD值
        OLED_ShowNum(2, 9, Voltage, 1);     // 显示电压值的整数部分
        OLED_ShowNum(2, 11, (uint16_t)(Voltage * 100) % 100, 2);    // 显示电压值的小数部分

        Delay_ms(100);
    }
}
```

（4）代码完成后，编译并下载到硬件中执行，会发现实验结果是随着电位器的调节，OLED能够显示正确的数字电压值，符合预期设计。

2、AD单通道连续转换

（1）实验目的

通过控制STM32读取因**光敏传感器**变化引起输入电压变化模拟量，在OLED上显示AD转换后的数字值。

（2）程序设计

3、AD单通道连续转换-模拟看门狗-触发灯亮

（1）实验目的

通过控制STM32读取因**光敏传感器**变化引起输入电压变化模拟量，在OLED上显示AD转换后的数字值。通过模拟看门狗阈值控制LED灯的亮灭。

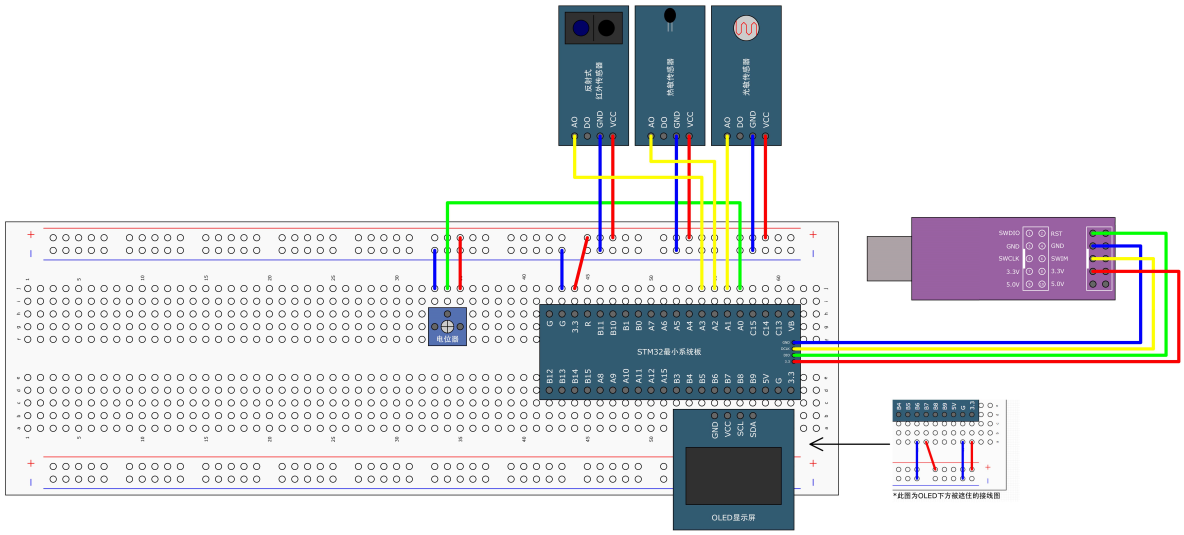
（2）程序设计

4、AD多通道

（1）实验目的

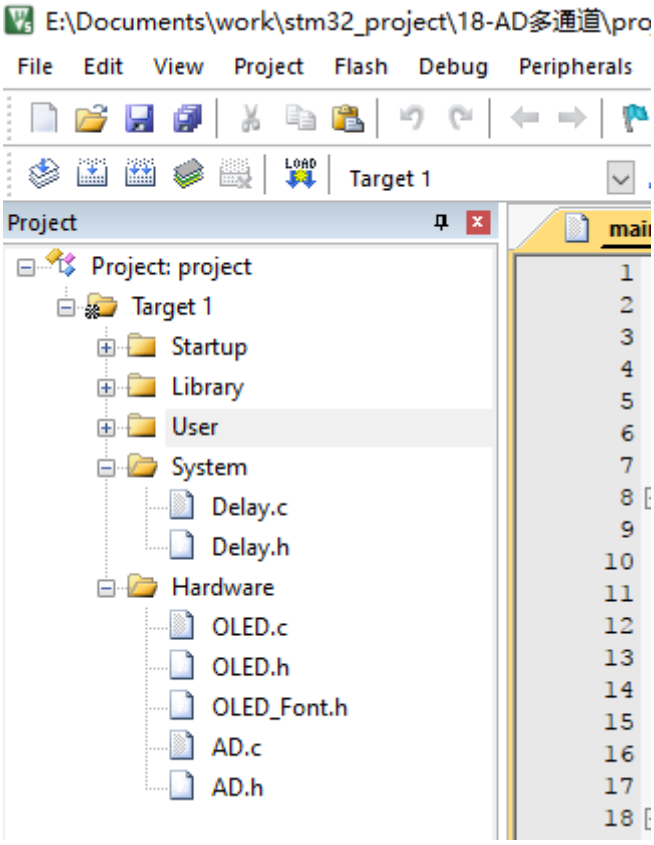
通过控制STM32同时读取4路电压输入，分别是电位器、光敏传感器、热敏传感器以及反射式红外传感器，利用AD转换，将多通道的模拟量转换为对应的数字量显示到OLED上。

（2）硬件电路接线图



（3）程序设计

工程结构如下图：



其中Hardware组与之前工程相比，这个组主要增加了AD相关驱动代码适配多通道有所调整。具体代码展示如下：

AD.c

```
#include "stm32f10x.h"                  // Device header

void AD_Init(void) {
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_ADC1, ENABLE);    // 打开ADC1的时钟
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);   // 打开GPIOA的时钟

    RCC_ADCCLKConfig(RCC_PCLK2_Div6);   // 配置ADC时钟，选择时钟6分频

    GPIO_InitTypeDef GPIO_InitStructure;
```

```
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AIN;   // 配置模拟输入模式
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_0 | GPIO_Pin_1 | GPIO_Pin_2 | GPIO_Pin_3;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    ADC_InitTypeDef ADC_InitStructure;
    ADC_InitStructure.ADC_ContinuousConvMode = DISABLE;
    ADC_InitStructure.ADC_DataAlign = ADC_DataAlign_Right;
    ADC_InitStructure.ADC_ExternalTrigConv = ADC_ExternalTrigConv_None;
    ADC_InitStructure.ADC_Mode = ADC_Mode_Independent;
    ADC_InitStructure.ADC_NbrOfChannel = 1;
    ADC_InitStructure.ADC_ScanConvMode = DISABLE;
    ADC_Init(ADC1, &ADC_InitStructure);     // 配置ADC初始化

    ADC_Cmd(ADC1, ENABLE);      // 使能ADC

    // ADC校准
    ADC_ResetCalibration(ADC1);
    while (ADC_GetResetCalibrationStatus(ADC1) == SET);
    ADC_StartCalibration(ADC1);
    while (ADC_GetCalibrationStatus(ADC1) == SET);
}

uint16_t AD_GetValue(uint8_t ADC_Channel) {
    //在每次转换前，根据函数形参灵活更改规则组的通道1
    ADC_RegularChannelConfig(ADC1, ADC_Channel, 1, ADC_SampleTime_55Cycles5);
    ADC_SoftwareStartConvCmd(ADC1, ENABLE);     // 软件触发AD转换一次
    while (ADC_GetFlagStatus(ADC1, ADC_FLAG_EOC) == RESET);     //等待EOC标志位，即等待AD转
换结束
    return ADC_GetConversionValue(ADC1);    // 读数据寄存器，得到AD转换的结果
}
```

AD.h

```
#ifndef __AD_H
#define __AD_H

void AD_Init(void);
uint16_t AD_GetValue(uint8_t ADC_Channel);

#endif
```

main.c

```
#include "stm32f10x.h"                  // Device header
#include "Delay.h"
#include "OLED.h"
#include "AD.h"
```

```c
uint16_t AD0, AD1, AD2, AD3;

int main(void){

    OLED_Init();
    AD_Init();

    OLED_ShowString(1, 1, "AD0:");
    OLED_ShowString(2, 1, "AD1:");
    OLED_ShowString(3, 1, "AD2:");
    OLED_ShowString(4, 1, "AD3:");

    while(1){
        AD0 = AD_GetValue(ADC_Channel_0);    // 单次启动ADC，转换通道0
        AD1 = AD_GetValue(ADC_Channel_1);    // 单次启动ADC，转换通道1
        AD2 = AD_GetValue(ADC_Channel_2);    // 单次启动ADC，转换通道2
        AD3 = AD_GetValue(ADC_Channel_3);    // 单次启动ADC，转换通道3

        OLED_ShowNum(1, 5, AD0, 4);
        OLED_ShowNum(2, 5, AD1, 4);
        OLED_ShowNum(3, 5, AD2, 4);
        OLED_ShowNum(4, 5, AD3, 4);

        Delay_ms(100);
    }
}
```

（4）代码完成后，编译并下载到硬件中执行，会发现实验结果是任意一路的输入电压模拟量发生变化，都可以通过AD转换将其数字量显示到OLED上，符合预期设计。

三、DMA

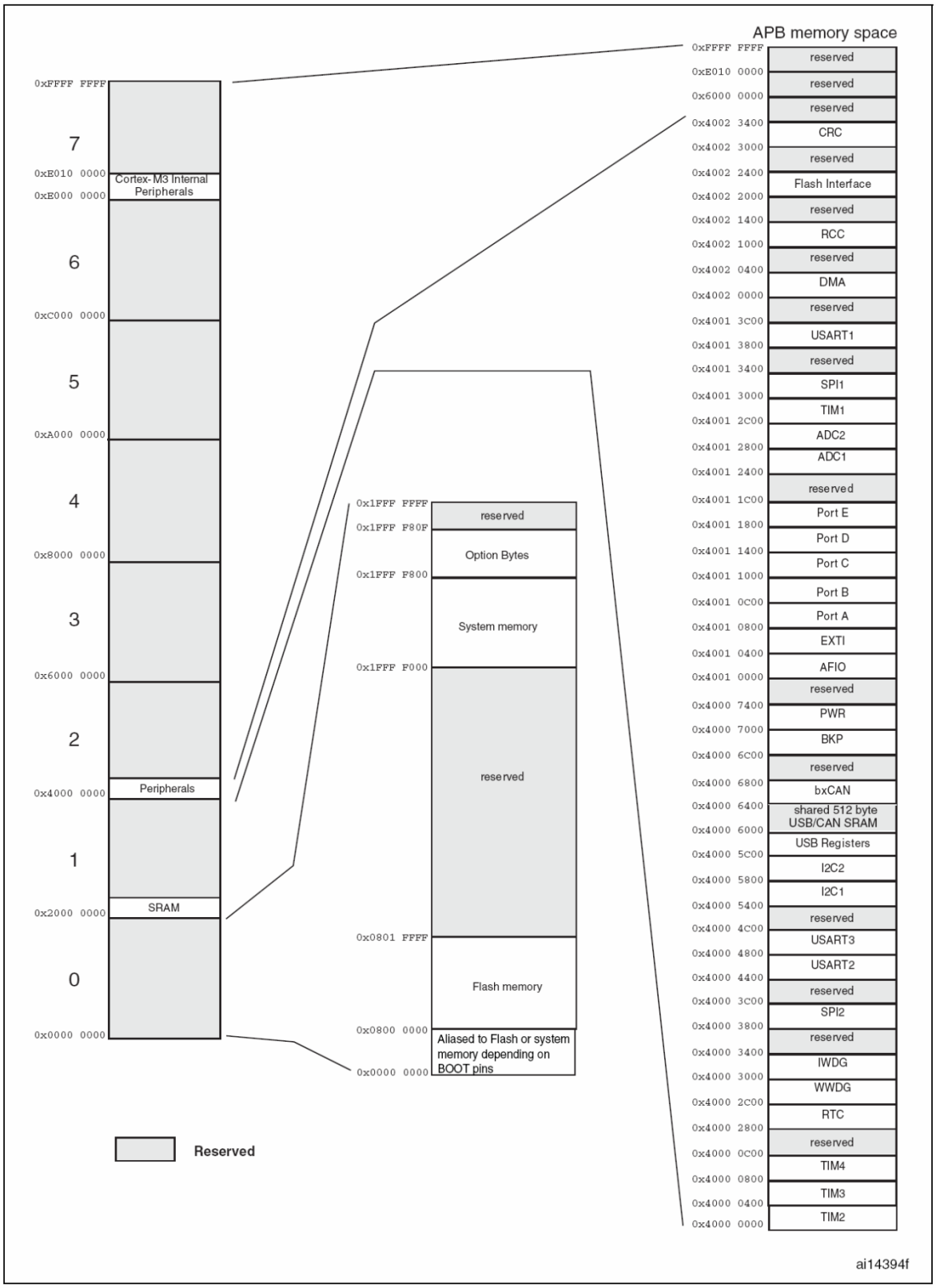
1、基础介绍

- DMA（Direct Memory Access）直接存储器存取，无须CPU干预，节省了CPU的资源
- 外设（外设寄存器）和存储器或者存储器和存储器之间的高速数据传输
    - 存储器：
        - SRAM -- 可读可写，掉电丢失
        - 程序存储器Flash - 只读（也不是绝对的，设置后可以写），掉电不丢失
- 12个独立可配置的通道
    - DMA1（7个通道）：STM32F103C8T6芯片的DMA资源只有DMA1（7个通道）
    - DMA2（5个通道）
- 触发
    - 支持软件触发
    - 特定的硬件触发

2、存储器映像

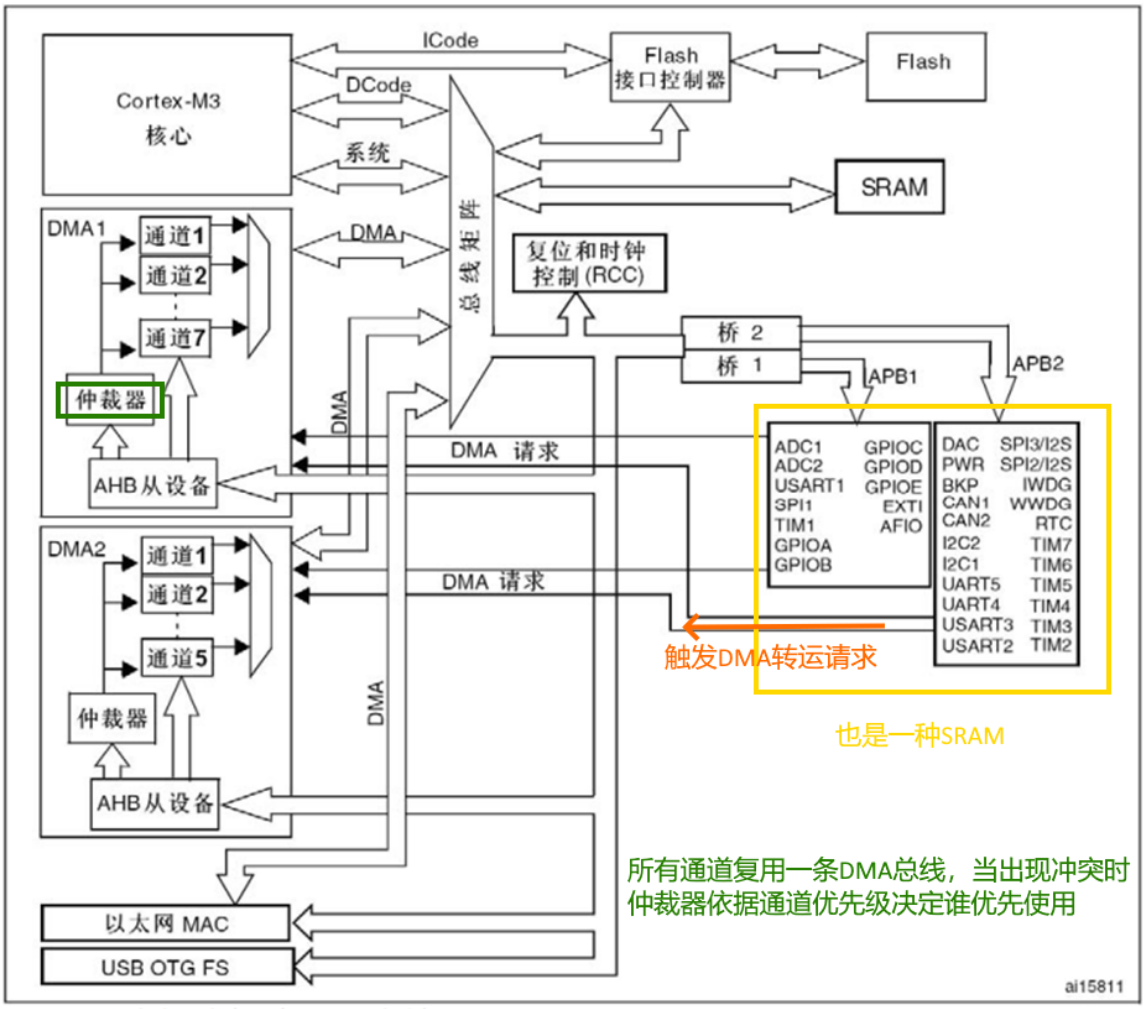
STM32F103C8T6的FLASH是**64**K，SRAM是**20K**。

类型	起始地址	存储器	用途
ROM	0x0800 0000	程序存储器Flash	存储C语言编译后的程序代码
0x1FFF F000	系统存储器	存储BootLoader，用于串口下载	
0x1FFF F800	选项字节	存储一些独立于程序代码的配置参数	
RAM	0x2000 0000	运行内存SRAM	存储运行过程中的临时变量
0x4000 0000	外设寄存器	存储各个外设的配置参数	
0xE000 0000	内核外设寄存器	存储内核各个外设的配置参数	

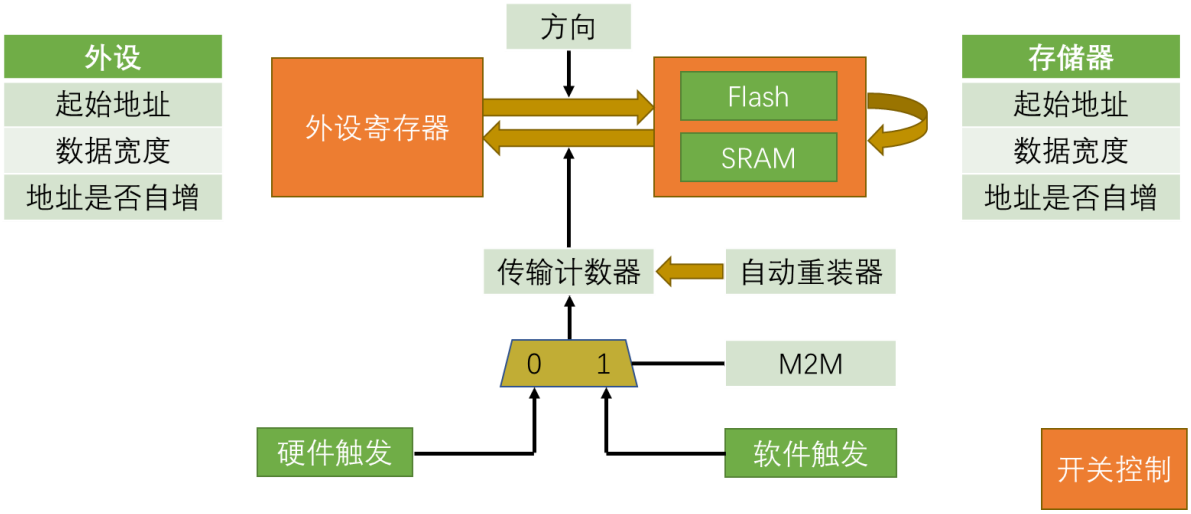

APB memory space
0xFFFF FFFF
7
0xE010 0000
Cortex-M3 Internal Peripherals
0xE000 0000
6
0xC000 0000
5
0xA000 0000
4
0x8000 0000
3
0x6000 0000
2
0x4000 0000
Peripherals
1
0x2000 0000
SRAM
0
0x0000 0000
0x1FFF FFFF
reserved
0x1FFF F80F
Option Bytes
0x1FFF F800
System memory
0x1FFF F000
reserved
0x0801 FFFF
Flash memory
0x0800 0000
Aliased to Flash or system memory depending on BOOT pins
0x0000 0000
Reserved
0xE010 0000
reserved
0x6000 0000
reserved
0x4002 3400
CRC
0x4002 3000
reserved
0x4002 2400
Flash Interface
0x4002 2000
reserved
0x4002 1400
RCC
0x4002 1000
reserved
0x4002 0400
DMA
0x4002 0000
reserved
0x4001 3C00
USART1
0x4001 3800
reserved
0x4001 3400
SPI1
0x4001 3000
TIM1
0x4001 2C00
ADC2
0x4001 2800
ADC1
0x4001 2400
reserved
0x4001 1C00
Port E
0x4001 1800
Port D
0x4001 1400
Port C
0x4001 1000
Port B
0x4001 0C00
Port A
0x4001 0800
EXTI
0x4001 0400
AFIO
0x4001 0000
reserved
0x4000 7400
PWR
0x4000 7000
BKP
0x4000 6C00
reserved
0x4000 6800
bxCAN
0x4000 6400
shared 512 byte USB/CAN SRAM
0x4000 6000
USB Registers
0x4000 5C00
I2C2
0x4000 5800
I2C1
0x4000 5400
reserved
0x4000 4C00
USART3
0x4000 4800
USART2
0x4000 4400
reserved
0x4000 3C00
SPI2
0x4000 3800
reserved
0x4000 3400
IWDG
0x4000 3000
WWDG
0x4000 2C00
RTC
0x4000 2800
reserved
0x4000 0C00
TIM4
0x4000 0800
TIM3
0x4000 0400
TIM2
0x4000 0000
ai14394f

3、DMA框图

图21　DMA框图



4、DMA基本结构



DMA基本结构图，图中包含数据转运的两大站点，左边是外设寄存器站点，右边是存储器站点包括Flash和SRAM。在STM32手册里所说的存储器一般是特指Flash和SRAM，不包含外设寄存器，外设寄存器一般被直接称作外设，因此通常是外设到存储器，存储器到存储器这样来描述的。寄存器也是存储器的一种，STM32使用外设和存储器来作区分只是描述方法的不同而已。DMA的数据转运可以是从外设到存储器，也是可以从存储器到外设。具体是向左还是向右，有一个方向的参数可以进行控制。此外，还有一种转运方式是存储器到存储器，比如Flash到SRAM，或者SRAM到SRAM。由于Flash是只读的，所以DMA不可以进行SRAM到Flash，或者Flash到Flash的转运操作。要进行数据转运需要指定从哪里转到哪里，因此在外设和存储器两个站点都有三个参数。



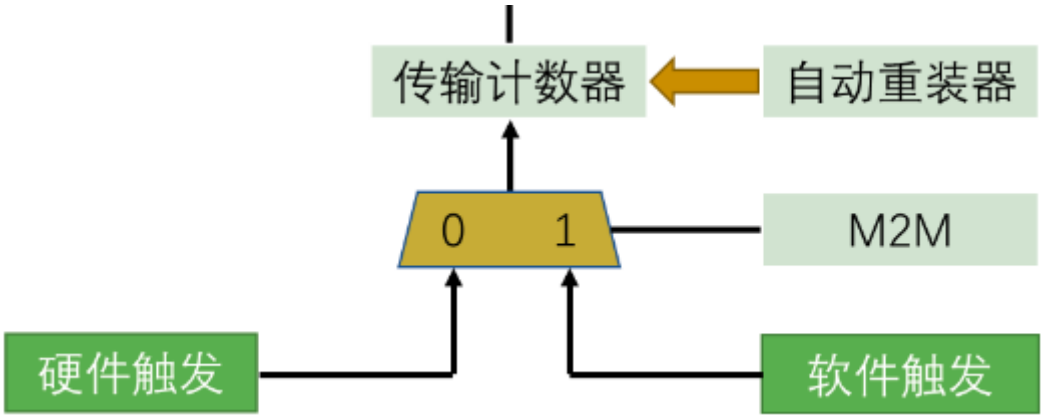
第一个是起始地址，有外设端的起始地址
和存储器端的起始地址，这两个参数决定数据是从哪里来，到哪里去的。

第二个参数是数据宽度，这个参数的作用是指定一次转运要按多大的数据宽度来进行，它可以选择字节byte、半字half word和字word。字节就是8位，也就是一次转运一个uint8_t这么大的数据，半字是16位，就是一次转运一个uint16_t这么大。字是32位，就是一次转运uint32_t这么大。比如转运ADC的数据，ADC的结果是uint16_t这么大，因此这个参数就要选择半字，一次转运一个unit6_t。

第三个参数是地址是否自增，这个参数的作用是指定一次转运完成后，下一次转运是不是要把地址移动到下一个位置去，这就相当于是指针p++。比如，ADC扫描模式用DMA进行数据转运，外设地址是ADC_DR寄存器。寄存器地址是不用自增的。如果自增，那下一次转运就跑到别的寄存器那里去了。而存储器地址就需要自增，每转运一个数据后就往后挪，要不然下次再转就把上次的覆盖掉了，这就是地址是否自增的作用。

如果要进行存储器到存储器的数据转运，那么就需要把其中一个存储器的地址放在外设的这个站点，这样就能进行存储器到存储器的转运了。这个站点虽然叫外设存储器，但它就只是个名字而已。并不是说这个地址只能写外设存储器的地址，这个是没有限制的。甚至你可以在外设站点写存储器的地址，存储器站点写外设的地址然后方向参数给反过来，这样也是可以的。

下面再看传输计数器，它就是用来指定总共需要转运几次的。



传输计数器是一个自减计数器，比如给它写入5，那DMA就只能进行5次数据转运。转运过程中每转运一次，计数器的数就会减1，当传输计数器减到0之后，DMA就不会再进行数据转运了。此外传输计数器减到0之后，之前站点自增的地址也会恢复到起始地址的位置，以方便之后DMA开始新一轮的转换。在传输计数器的右边有一个自动重装器，这个自动重装器的作用就是传输计数器减到0之后，是否要自动恢复到最初的值。比如最初传输计数器给5，如果不使用自动重装器，那转运5次后DMA就结束了。如果使用自动重装器，那转运5次计数器减到0后，就会立即重装到初始值5。这个就是自动重装器，它决定了转运的模式。如果不重装，就是正常的单次模式，如果重装，就是循环模式。比如如果想转运一个宿主，那一般就是单次模式转运一轮就结束了。如果是ADC扫描模式加连续转换，那为了配合ADC，DMA也需要使用循环模式。所以这个循环模式和ADC的连续模式差不多，都是指定一轮工作完成后，是不是立即开始下一轮工作。

再看DMA的触发控制，触发就是决定DMA需要在什么时机进行转运的触发源，有硬件触发和软件触发。具体选择哪个由M2M这个参数决定，M2M就是memory to memory，所以M2M就是存储器到存储器。当我们给M2M位给1时，DMA就会选择软件触发。这个软件触发并不是调用某个函数一次触发一次。软件触发的执行逻辑是以最快的速度，连续不断地触发，DMA争取早日把传输计数器清零完成这一轮的转换。所以这里的软件触发不同于之前外部中断和ADC的软件触发，在这里可以把它理解成连续触发。软件触发和循环模式不能同时用，因为软件触发就是想把传输计数器清零，循环模式是清零后自动重装，如果同时用的话，DMA就停不下来了。软件触发一般适用于存储器到存储器的转运，因为存储器到存储器的转运是软件启动，不需要时机，并且想尽快完成的任务。当M2M位给0时，就是使用硬件触发。硬件触发源可以选择ADC、串口、定时器等等。使用硬件触发的转运一般都是与外设有关的转运。这些转运需要一定的时机，比如ADC转换完成，串口收到数据，定时时间到等等。所以需要使用硬件触发，在硬件达到这些时机时，传个信号过来来触发DMA进行转运，这就是硬件触发。

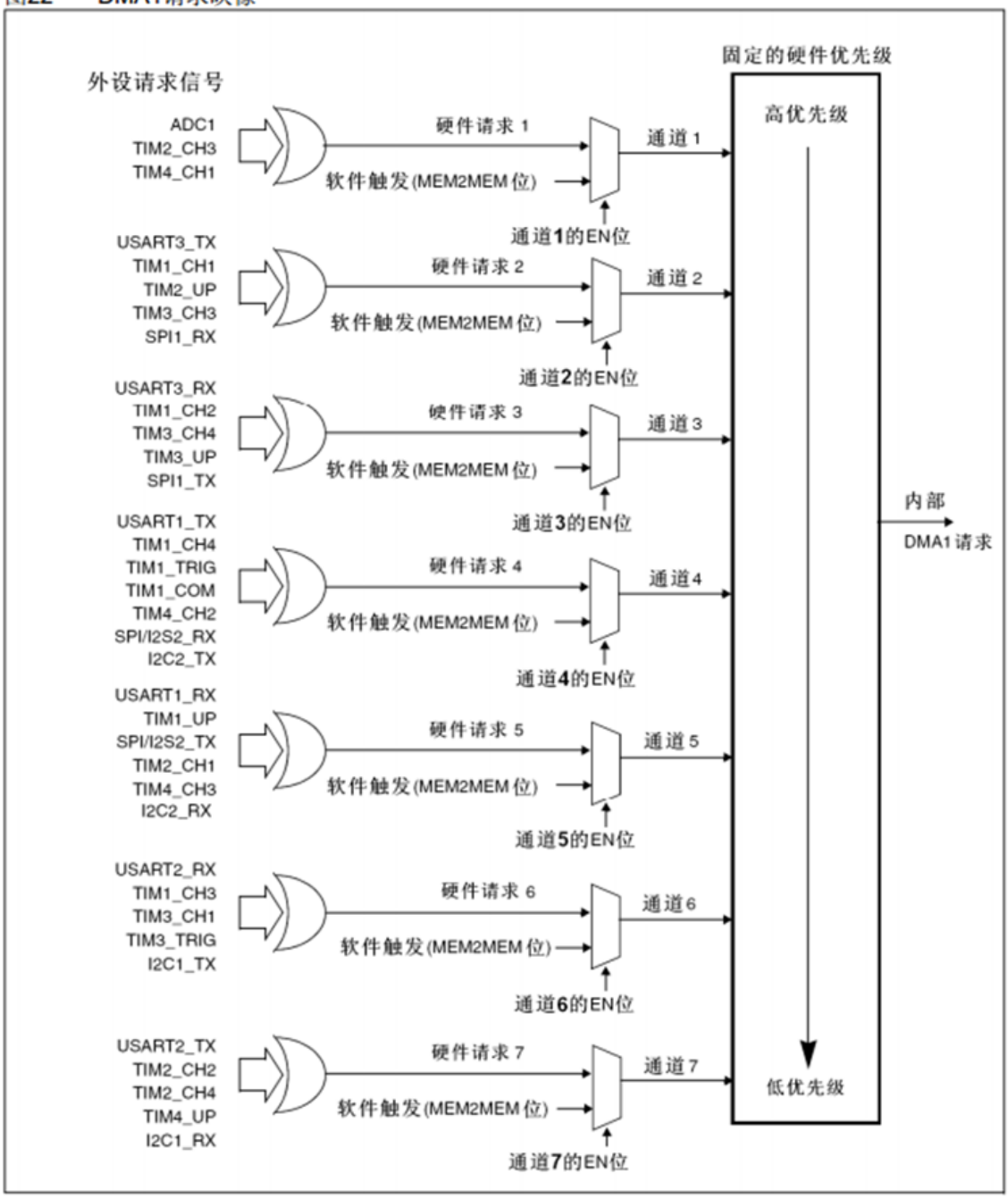
图中还有一个开关控制，它就是DMA_CMD函数。



当给DMA使能后，DMA就准备就绪可以进行转运了。
DMA进行转运有几个条件：第一就是开关控制DMA_CMD必须使能；第二就是传输计数器必须大于0；第三就是触发源必须有触发信号。触发一次，转运一次，传输计数器自检一次。当传输计数器等于0，且没有自动重装时，这时无论是否触发DMA，都不会再进行转运了。此时就需要DMA_CMD给Disable关闭DMA，再为传输计数器写一个大于0的数，再DMA_CMD给Enable开启DMA，DMA才能继续工作。需要注意写传输计数器时，必须要先关闭DMA再进行，不能在DMA开启时写传输计数器。

5、DMA请求

图22　DMA1请求映像

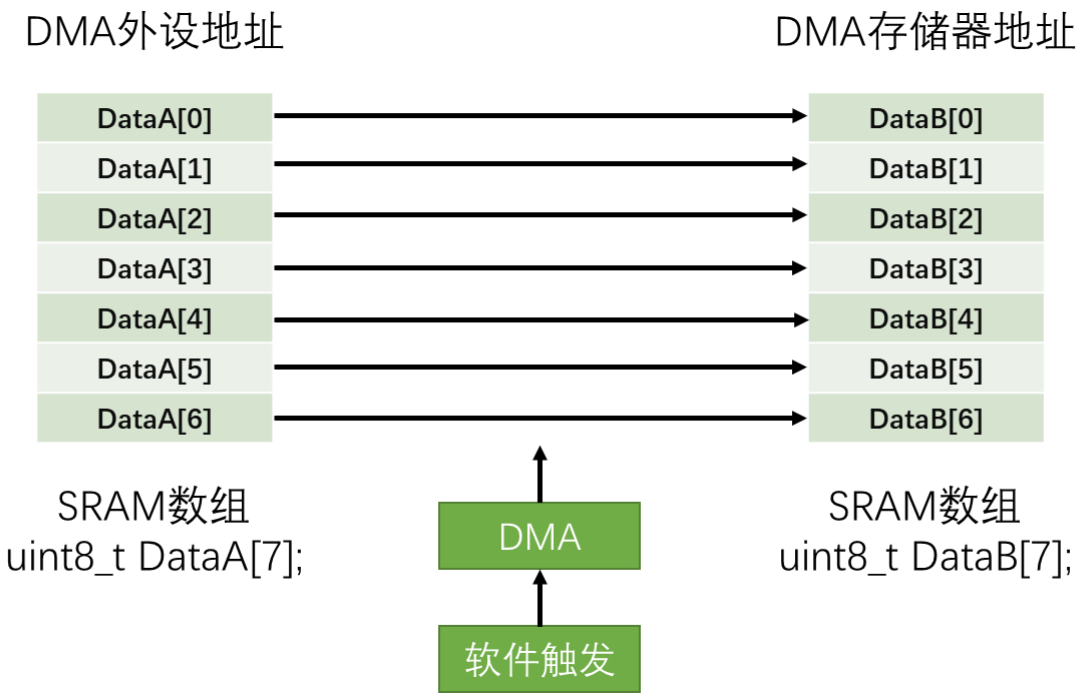


6、数据宽度与对齐

表57　可编程的数据传输宽度和大小端操作(当PINC = MINC = 1)

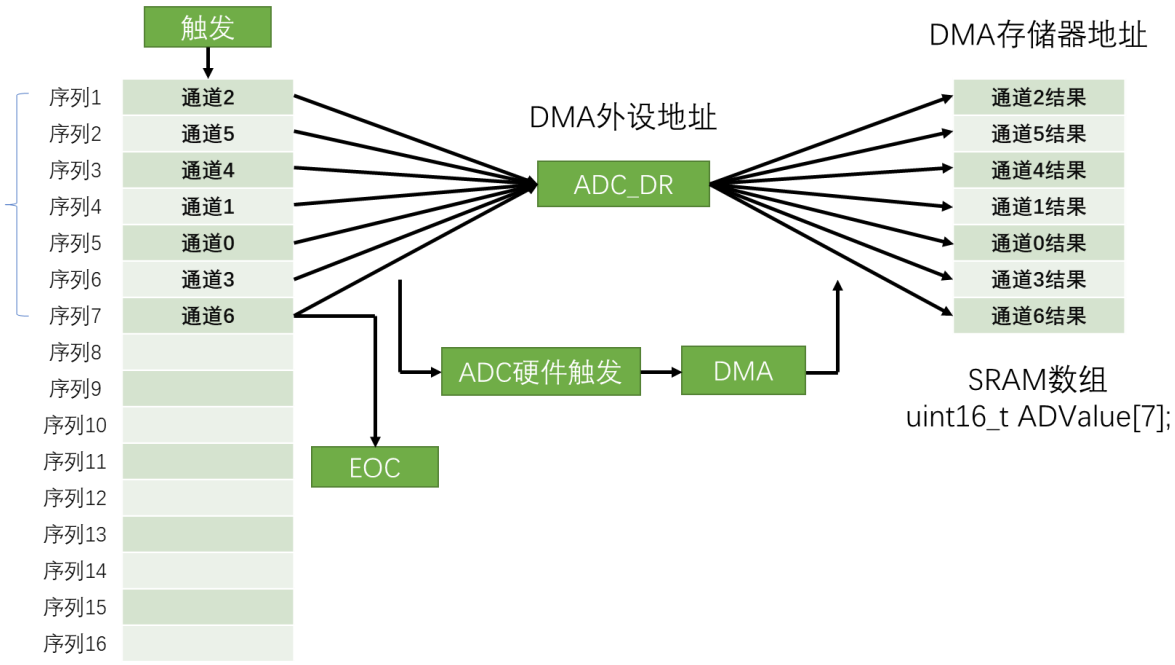
源端宽度	目标宽度	传输数目	源：地址/数据	传输操作	目标：地址/数据
8	8	4	0x0 / B0 0x1 / B1 0x2 / B2 0x3 / B3	1: 在0x0读B0[7:0]，在0x0写B0[7:0] 2: 在0x1读B1[7:0]，在0x1写B1[7:0] 3: 在0x2读B2[7:0]，在0x2写B2[7:0] 4: 在0x3读B3[7:0]，在0x3写B3[7:0]	0x0 / B0 0x1 / B1 0x2 / B2 0x3 / B3
8	16	4	0x0 / B0 0x1 / B1 0x2 / B2 0x3 / B3	1: 在0x0读B0[7:0]，在0x0写00B0[15:0] 2: 在0x1读B1[7:0]，在0x2写00B1[15:0] 3: 在0x2读B2[7:0]，在0x4写00B2[15:0] 4: 在0x3读B3[7:0]，在0x6写00B3[15:0]	0x0 / 00B0 0x2 / 00B1 0x4 / 00B2 0x6 / 00B3
8	32	4	0x0 / B0 0x1 / B1 0x2 / B2 0x3 / B3	1: 在0x0读B0[7:0]，在0x0写000000B0[31:0] 2: 在0x1读B1[7:0]，在0x4写000000B1[31:0] 3: 在0x2读B2[7:0]，在0x8写000000B2[31:0] 4: 在0x3读B3[7:0]，在0xC写000000B3[31:0]	0x0 / 000000B0 0x4 / 000000B1 0x8 / 000000B2 0xC / 000000B3
16	8	4	0x0 / B1B0 0x2 / B3B2 0x4 / B5B4 0x6 / B7B6	1: 在0x0读B1B0[15:0]，在0x0写B0[7:0] 2: 在0x2读B3B2[15:0]，在0x1写B2[7:0] 3: 在0x4读B5B4[15:0]，在0x2写B4[7:0] 4: 在0x6读B7B6[15:0]，在0x3写B6[7:0]	0x0 / B0 0x1 / B2 0x2 / B4 0x3 / B6
16	16	4	0x0 / B1B0 0x2 / B3B2 0x4 / B5B4 0x6 / B7B6	1: 在0x0读B1B0[15:0]，在0x0写B1B0[15:0] 2: 在0x2读B3B2[15:0]，在0x2写B3B2[15:0] 3: 在0x4读B5B4[15:0]，在0x4写B5B4[15:0] 4: 在0x6读B7B6[15:0]，在0x6写B7B6[15:0]	0x0 / B1B0 0x2 / B3B2 0x4 / B5B4 0x6 / B7B6
16	32	4	0x0 / B1B0 0x2 / B3B2 0x4 / B5B4 0x6 / B7B6	1: 在0x0读B1B0[15:0]，在0x0写0000B1B0[31:0] 2: 在0x2读B3B2[15:0]，在0x4写0000B3B2[31:0] 3: 在0x4读B5B4[15:0]，在0x8写0000B5B4[31:0] 4: 在0x6读B7B6[15:0]，在0xC写0000B7B6[31:0]	0x0 / 0000B1B0 0x4 / 0000B3B2 0x8 / 0000B5B4 0xC / 0000B7B6
32	8	4	0x0 / B3B2B1B0 0x4 / B7B6B5B4 0x8 / BBBAB9B8 0xC / BFBEBDBC	1: 在0x0读B3B2B1B0[31:0]，在0x0写B0[7:0] 2: 在0x4读B7B6B5B4[31:0]，在0x1写B4[7:0] 3: 在0x8读BBBAB9B8[31:0]，在0x2写B8[7:0] 4: 在0xC读BFBEBDBC[31:0]，在0x3写BC[7:0]	0x0 / B0 0x1 / B4 0x2 / B8 0x3 / BC
32	16	4	0x0 / B3B2B1B0 0x4 / B7B6B5B4 0x8 / BBBAB9B8 0xC / BFBEBDBC	1: 在0x0读B3B2B1B0[31:0]，在0x0写B1B0[15:0] 2: 在0x4读B7B6B5B4[31:0]，在0x2写B5B4[15:0] 3: 在0x8读BBBAB9B8[31:0]，在0x4写B9B8[15:0] 4: 在0xC读BFBEBDBC[31:0]，在0x6写BDBC[15:0]	0x0 / B1B0 0x2 / B5B4 0x4 / B9B8 0x6 / BDBC
32	32	4	0x0 / B3B2B1B0 0x4 / B7B6B5B4 0x8 / BBBAB9B8 0xC / BFBEBDBC	1: 在0x0读B3B2B1B0[31:0]，在0x0写B3B2B1B0[31:0] 2: 在0x4读B7B6B5B4[31:0]，在0x4写B7B6B5B4[31:0] 3: 在0x8读BBBAB9B8[31:0]，在0x8写BBBAB9B8[31:0] 4: 在0xC读BFBEBDBC[31:0]，在0xC写BFBEBDBC[31:0]	0x0 / B3B2B1B0 0x4 / B7B6B5B4 0x8 / BBBAB9B8 0xC / BFBEBDBC

7、数据转运+DMA



这个图中描述的任务是将SRAM里的数组DataA转运到另一个数组DataB中。在这种情况下其基本结构的各个参数该如何配置呢？首先要考虑的是外设站点和存储器站点的起始地址、数据宽度、地址是否自增三个参数。外设地址应该填DataA数组的首地址，存储器地址给DataB数组的首地址。由于两个数组的类型都是uint8_t，所以数据宽度都是按8位的字节传出。地址是否自增呢？这里想要的效果是DataA[0]转到DataB[0]， DataA[1]转到DataB[1]等等，两个数值的位置一一对应，所以转运完DataA[0]和DataB[0]之后，两个站点的地址都应该自增，都移动到下一个数据的位置，继续转运DataA[1]和DataB[1]，这样来进行。如果你左边不自增，右边自增，效果就是转运完成后，DataB的所有数据都会等于DataA[0]。如果左边自增，右边不自增，那效果就是转运完成后，DataB[0]等于DataA的最后一个数，DataB其他的数不变。如果左右都不自增，那就一直是DataA[0]转到DataB[0]，其他的数据不变，这就是地址是否自增的效果。关于方向参数，显然是外设站点转运到存储器站点。如果你想把DataB的数据转运到DataA，可以把方向参数换过来，这样就是反向转运了。这里数组有7个元素，要转运7次，所以传输计数器给7，自动重装暂时不需要。由于是存储器到存储器的数据转运，不需要等待硬件时机，尽快转运完成就行了，因此使用软件触发。最后调用DMA_CMD给DMA使能，这样数据就会从DataA转运到DataB，转运7次之后，传输计数器自减到0，DMA停止转运。这里的数据转运是一种复制转运，转运完成后DataA的数据并不会消失，这个过程相当于是把DataA的数据复制到了DataB的位置。

8、ADC扫描模式+DMA



这个图的任务是ADC扫描模式加DMA。左边是ADC扫描模式的执行流程，有7个通道，触发一次后，7个通道依次进行AD转换，然后转换结果都放到ADC_DR数据寄存器里面。这里要做的是在每个单独的通道转换完成后，进行一次DMA数据转运，并且目的地址进行自增，这样数据就不会被覆盖了。因此DMA的配置就是外设地址写作ADC_DR这个寄存器的地址，存储器的地址可以在SRAM中定一个宿主ADValue，然后把ADValue的地址当作存储器的地址。数据宽度因为ADC_DR和SRAM数组，都是uint16_t的数据，所以数据宽度都是16位的半字传输。从图中可以看出外设地址不自增，存储器地址需要自增。传输方向是外设站点到存储器站点。有7个通道，所以传输计数器计数7次。ADC的配置如果是单次扫描，那DMA的传输计数器可以不自动重装，转换一轮就停止。如果ADC的配置是连续扫描，那DMA就可以使用自动重装，在ADC启动下一轮转换的时候，DMA也启动下一轮的转运，ADC和DMA同步工作。关于触发选择，这里ADC_DR的值是在ADC单个通道转换完成后才会有效，所以DMA转运的时机需要和ADC单个通道转换完成同步，所以DMA的触发要选择ADC的硬件触发。对于硬件触发来说这里要说明一下，ADC扫描模式在每个单独的通道转换完成后，没有任何标志位，也不会触发中断，所以程序不太好判断某一个通道转换完成的时机是什么时候。但是虽然单个通道转换完成后，不产生任何标志位和中断，但是它应该会产生DMA请求，去触发DMA转运。这部分内容手册里并没有详细描述，主要是根据实际实验，单个通道的DMA请求肯定是有的。以上就是ADC扫描模式和DMA配合使用的流程。

一般来说DMA最常见的用途就是配合ADC的扫描模式。因为ADC扫描模式有个数据覆盖的特征，或者可以说这个数据覆盖的问题是ADC固有的缺陷。这个缺陷使ADC和DMA成了最常见的伙伴，ADC对DMA的需求是非常强烈的。

四、DMA实验

1、DMA拷贝数据

（1）实验目的

利用DMA把arrA[] 拷贝到 arrB[] ,每拷贝一次arrA中的值都自动变化一次

（2）程序设计

MyDMA.c

```
#include "stm32f10x.h"                  // Device header

uint16_t DataLength = 0;
void MyDMA_Init(uint32_t Aaddr,uint32_t Baddr,uint16_t lenght)
{
    DataLength = lenght;
    //1. DMA是一个设备，需要开时钟，AHB。
    RCC_AHBPeriphClockCmd(RCC_AHBPeriph_DMA1,ENABLE);
    //2. DMA的初始化
    DMA_InitTypeDef DMA_InitTypeStructure;

    DMA_InitTypeStructure.DMA_PeripheralBaseAddr = Aaddr; // 拷贝的源
    DMA_InitTypeStructure.DMA_PeripheralDataSize = DMA_PeripheralDataSize_Byte; //每个数据是1个字节大小
    DMA_InitTypeStructure.DMA_PeripheralInc = DMA_PeripheralInc_Enable;//自增
    DMA_InitTypeStructure.DMA_MemoryBaseAddr = Baddr;  //拷贝的目标
    DMA_InitTypeStructure.DMA_MemoryDataSize = DMA_MemoryDataSize_Byte; //和源数据每个元素长度一样
    DMA_InitTypeStructure.DMA_MemoryInc = DMA_MemoryInc_Enable; //自增
    DMA_InitTypeStructure.DMA_BufferSize = lenght; //传送多少个
    DMA_InitTypeStructure.DMA_DIR = DMA_DIR_PeripheralSRC; //Peripheral 是源
    DMA_InitTypeStructure.DMA_M2M = DMA_M2M_Enable; //软件触发，内存到内存的拷贝
    DMA_InitTypeStructure.DMA_Mode = DMA_Mode_Normal; // 当DMA_MemoryInc_Enable 的时候不能选择循环模式DMA_Mode_Circular
    DMA_InitTypeStructure.DMA_Priority = DMA_Priority_Medium; // 优先级

    DMA_Init(DMA1_Channel1,&DMA_InitTypeStructure);
    //3. DMA使能  -- 失能
    DMA_Cmd(DMA1_Channel1,DISABLE);
}

void MyDMA_Transfer(void)
{
    DMA_Cmd(DMA1_Channel1,DISABLE);  //每次设置DMA_SetCurrDataCounter 之前要失能
    //调用一次传送一次
    // 1.计数器重置
    DMA_SetCurrDataCounter(DMA1_Channel1,DataLength); //DataLength 填写要拷贝的数据长度
```

```
    // 2.DMA使能 --- 使能
    DMA_Cmd(DMA1_Channel1,ENABLE);

    // 3.等待DMA转运完成
    while(DMA_GetFlagStatus(DMA1_FLAG_TC1)==RESET); //等待拷贝完成

}
```

main.c

```
#include "stm32f10x.h"                  // Device header
#include "Delay.h"
#include "OLED.h"
#include "MyADC.h"
#include "MyDMA.h"

//利用DMA把arrA[] 拷贝到 arrB[] ,每拷贝一次arrA中的值都自动变化一次

uint8_t arrA[] = {0x00,0x01,0x02,0x003};
uint8_t arrB[] = {0x00,0x00,0x00,0x00};


int main(void)
{

    OLED_Init();
    MyDMA_Init((uint32_t)arrA,(uint32_t)arrB,4);
    OLED_ShowHexNum(3,1,(uint32_t)arrA,8);//显示A的地址
    OLED_ShowHexNum(4,1,(uint32_t)arrB,8);//显示B的地址

    while(1)
    {
        OLED_ShowNum(1,1,arrA[0],2);
        OLED_ShowNum(1,4,arrA[1],2);
        OLED_ShowNum(1,7,arrA[2],2);
        OLED_ShowNum(1,10,arrA[3],2);
        Delay_ms(1000);
        MyDMA_Transfer();
        OLED_ShowNum(2,1,arrB[0],2);
        OLED_ShowNum(2,4,arrB[1],2);
        OLED_ShowNum(2,7,arrB[2],2);
        OLED_ShowNum(2,10,arrB[3],2);
        arrA[0]++;
        arrA[1]++;
        arrA[2]++;
        arrA[3]++;
        Delay_ms(1000);


    }
```

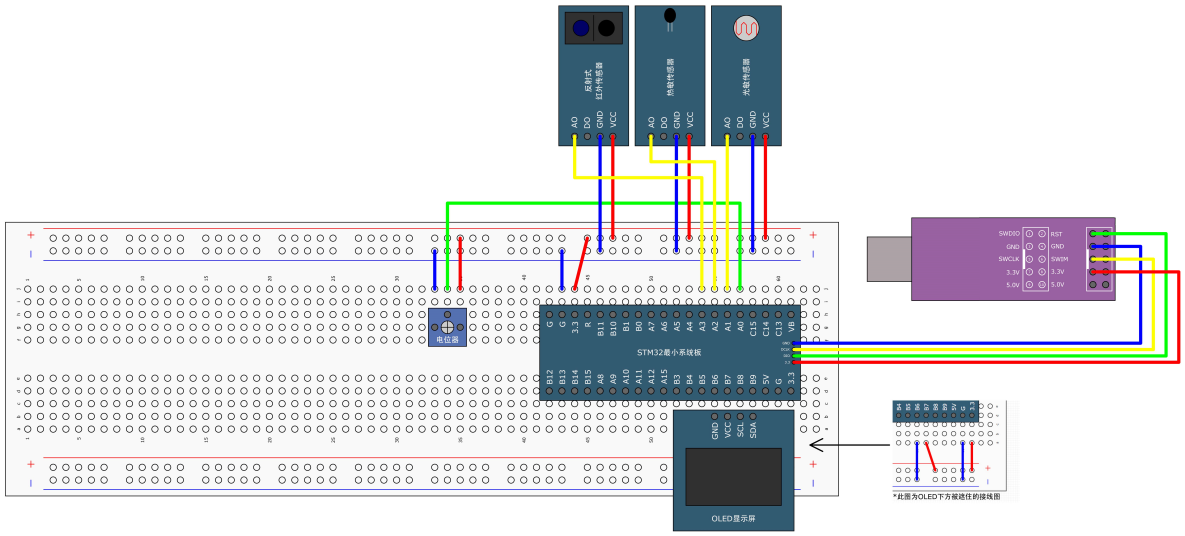
```
}
```

2、多通道的模拟量DMA

（1）实验目的

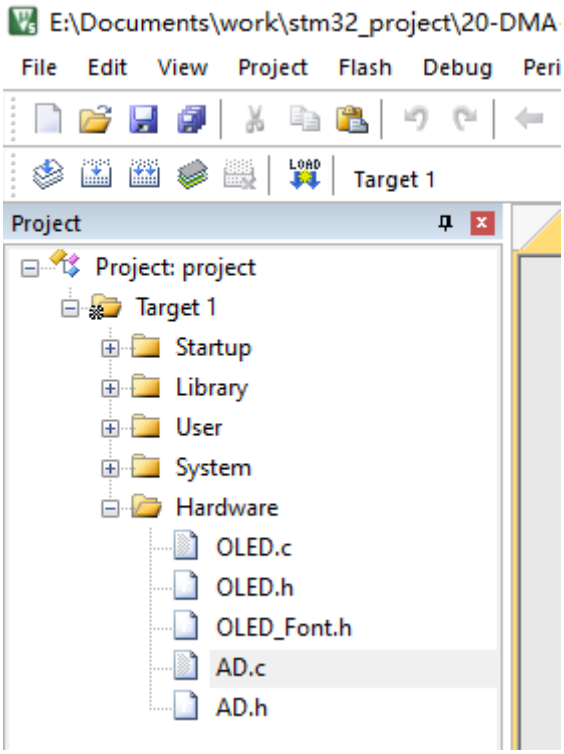
通过控制STM32同时读取4路电压输入，分别是电位器、光敏传感器、热敏传感器以及反射式红外传感器，利用DMA+AD转换，将多通道的模拟量转换为对应的数字量显示到OLED上。

（2）硬件电路接线图



（3）程序设计

工程结构如下图：



其中Hardware组与之前工程不一样，这个组主要增加了AD相关驱动代码。具体代码展示如下：

AD.c

```
#include "stm32f10x.h"                  // Device header
```

```
uint16_t AD_Value[4];

void AD_Init(void) {
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_ADC1, ENABLE);        // 打开ADC1的时钟
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);       // 打开GPIOA的时钟
    RCC_AHBPeriphClockCmd(RCC_AHBPeriph_DMA1, ENABLE);          // 打开DMA1的时钟

    RCC_ADCCLKConfig(RCC_PCLK2_Div6);   // 配置ADC时钟，选择时钟6分频

    GPIO_InitTypeDef GPIO_InitStructure;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AIN;   // 配置模拟输入模式
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_0 | GPIO_Pin_1 | GPIO_Pin_2 | GPIO_Pin_3;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    // 规则组序列1的位置，配置为通道0
    ADC_RegularChannelConfig(ADC1, ADC_Channel_0, 1, ADC_SampleTime_55Cycles5);
    // 规则组序列2的位置，配置为通道1
    ADC_RegularChannelConfig(ADC1, ADC_Channel_1, 2, ADC_SampleTime_55Cycles5);
    // 规则组序列3的位置，配置为通道2
    ADC_RegularChannelConfig(ADC1, ADC_Channel_2, 3, ADC_SampleTime_55Cycles5);
    // 规则组序列4的位置，配置为通道3
    ADC_RegularChannelConfig(ADC1, ADC_Channel_3, 4, ADC_SampleTime_55Cycles5);

    ADC_InitTypeDef ADC_InitStructure;
    ADC_InitStructure.ADC_ContinuousConvMode = ENABLE;  // 配置连续扫描
    ADC_InitStructure.ADC_DataAlign = ADC_DataAlign_Right;  // 配置数据右对齐
    ADC_InitStructure.ADC_ExternalTrigConv = ADC_ExternalTrigConv_None;     // 配置使用软
件触发
    ADC_InitStructure.ADC_Mode = ADC_Mode_Independent;      // 选择独立模式，单独使用ADC1
    ADC_InitStructure.ADC_NbrOfChannel = 4;     // 配置通道数，使用前4个通道
    ADC_InitStructure.ADC_ScanConvMode = ENABLE;    // 配置扫描模式
    ADC_Init(ADC1, &ADC_InitStructure);         // 配置ADC初始化

    DMA_InitTypeDef DMA_InitStructure;
    DMA_InitStructure.DMA_BufferSize = 4;   // 配置传输次数
    DMA_InitStructure.DMA_DIR = DMA_DIR_PeripheralSRC;  // 配置站点传输方向
    DMA_InitStructure.DMA_M2M = DMA_M2M_Disable;    // 配置硬件触发
    DMA_InitStructure.DMA_MemoryBaseAddr = (uint32_t)AD_Value;  // 配置存储器站点基地址
    DMA_InitStructure.DMA_MemoryDataSize = DMA_MemoryDataSize_HalfWord; // 配置存储器站点数
据传输宽度为字节
    DMA_InitStructure.DMA_MemoryInc = DMA_MemoryInc_Enable; // 配置存储器站点地址自增
//  DMA_InitStructure.DMA_Mode = DMA_Mode_Normal;   // 配置传输寄存器不自动重装
    DMA_InitStructure.DMA_Mode = DMA_Mode_Circular; // 配置传输寄存器自动重装
    DMA_InitStructure.DMA_PeripheralBaseAddr = (uint32_t)&ADC1->DR; // 配置外设站点基地址
    DMA_InitStructure.DMA_PeripheralDataSize = DMA_PeripheralDataSize_HalfWord; // 配置外
设站点数据传输宽度为半字
    DMA_InitStructure.DMA_PeripheralInc = DMA_PeripheralInc_Disable;    // 配置外设站点地址
不自增
    DMA_InitStructure.DMA_Priority = DMA_Priority_Medium;   // 配置中优先级
```

```
    DMA_Init(DMA1_Channel1, &DMA_InitStructure);    // DMA初始化

    DMA_Cmd(DMA1_Channel1, ENABLE);     // DMA1的通道1使能
    ADC_DMACmd(ADC1, ENABLE);       // ADC1触发DMA1的信号使能
    ADC_Cmd(ADC1, ENABLE);          // ADC1使能

    // ADC校准
    ADC_ResetCalibration(ADC1);
    while (ADC_GetResetCalibrationStatus(ADC1) == SET);
    ADC_StartCalibration(ADC1);
    while (ADC_GetCalibrationStatus(ADC1) == SET);

    // 软件触发ADC开始工作，ADC处于连续转换模式，触发一次后ADC就可以一直连续不断地工作
    ADC_SoftwareStartConvCmd(ADC1, ENABLE);
}
```

AD.h

```
#ifndef __AD_H
#define __AD_H

extern uint16_t AD_Value[4];
void AD_Init(void);

#endif
```

main.c

```
#include "stm32f10x.h"                  // Device header
#include "Delay.h"
#include "OLED.h"
#include "AD.h"

uint16_t AD0, AD1, AD2, AD3;

int main(void){

    OLED_Init();
    AD_Init();

    OLED_ShowString(1, 1, "AD0:");
    OLED_ShowString(2, 1, "AD1:");
    OLED_ShowString(3, 1, "AD2:");
    OLED_ShowString(4, 1, "AD3:");

    while(1){
        OLED_ShowNum(1, 5, AD_Value[0], 4);     // 显示转换结果第0个数据
        OLED_ShowNum(2, 5, AD_Value[1], 4);     // 显示转换结果第1个数据
        OLED_ShowNum(3, 5, AD_Value[2], 4);     // 显示转换结果第2个数据
```

```
        OLED_ShowNum(4, 5, AD_Value[3], 4);      // 显示转换结果第3个数据

        Delay_ms(100);      //延时100ms，增加转换的间隔时间
    }
}
```

（4）代码完成后，编译并下载到硬件中执行，会发现实验结果是是任意一路的输入电压模拟量发生变化，都可以通过AD转换将其数字量，然后再使用DMA转运数据最终将结果显示到OLED上，符合预期设计。

课程小结

1. ADC模数转换器可以将输入端口的模拟信号量转换为数字信号，其具有单次转换和连续转换，非扫描模式和扫描模式的组合，可以利用不同的模式对信号进行数字化转换。
2. DMA主要负责数据在外设和存储器之间的转运，利用DMA和ADC的配合可以很好的完成数据的存取。

随堂作业

1. 请问ADC转换与DMA是如何配合获取数据的?